iGAZE product implementation plan

Exhibition release 12 November 2026

Prepared for Muhammad Shahzeb Alam and the iGAZE team
Planning baseline: 7 November 2026, Asia/Karachi

Decision: build a dependable Windows assistive desktop product around intentional, context-aware navigation. Deliver a polished application, local profiles and activity storage, an offline installer, and a four-page product website. Preserve a measured path toward hybrid pointing after the exhibition.

The release promise

A person with restricted hand use can calibrate, read and scroll a document, switch browser tabs, control a presentation, and pause the system using an ordinary webcam. The app makes its current mode and pending action understandable and refuses to act when tracking is unreliable.

What makes this product distinctive

The proposed differentiation is a single interaction policy combining explicit intent, tracking-quality rejection, per-application commands, personal calibration and transparent recovery. This is a candidate research contribution to evaluate, not a verified world-first or an accuracy superiority claim.

Delivery layer	Commitment
Required before 12 Nov	Stable tracking and navigation; validated calibration; modern desktop UI; profiles and local data; reproducible build; Windows installer; Home, About, Features and Download website.
Conditional stretch	Large-target pointer with head refinement; limited personalization from independently labelled corrections; only if core release gates pass.
After 12 Nov	Full hybrid pointer, learned intent verifier, longitudinal drift evaluation, target-user study and publication rewrite.

Capacity assumption: one developer with AI assistance, roughly 8 focused hours per day on 8-11 November. The 32-hour baseline is aggressive; fallback rules on page 12 preserve a complete core product. Estimates are planning judgments, not guarantees.

How the recommendations translate into work

Supervisor documents are research inputs. Their suggested features and blue manuscript passages are proposals, not authorization to invent results or instructions to change the project now. This document plans implementation; no application refactor is included in this delivery.

Supervisor recommendation	Decision for exhibition	Evidence to retain
Quality gate and calibration validity	Required. Reject stale/no-face input; reject insufficient calibration; show a reason and retry.	Calibration sample counts, failure reason, valid-frame fraction and recovery events.
Robust pose and 1-Euro filtering	Required experiment with a small time box. Keep the proven baseline if the alternative loses stability.	Same replay samples; pitch jitter, lag and smile/talking sensitivity.
Hybrid gaze jump and head refinement	Prototype only after navigation and packaging pass. Large targets first.	Held-out target error and head-only vs hybrid movement/time.
Implicit calibration and replay buffer	Design hooks now; automatic online gaze learning deferred. Corrections are explicit labels.	Label source, region coverage, validation score and model rollback ID.
Personal false-activation verifier	Log accepted/cancelled actions now; learned verifier after exhibition.	Candidate events, genuine user labels and held-out sessions.
Configurable triggers and app profiles	Required reading and presentation profiles; optional wink only after calibration validation.	Profile/trigger configuration and action success checks.
Eyewear and motor-disability evaluation	Eyewear smoke checks before demo; formal target-user recruitment later with institutional guidance.	Group labels only when consented; protocol and participant-level outcomes.
Statistics and reproducibility corrections	Instrument now; audit old results after exhibition.	Exact software/model versions, task scripts, exports and raw observations.

The supervisor report describes LookAHead results from a nine-person pilot and recommends a hybrid pointer. Its detailed results remain supervisor-reported here: the original full text was not supplied in this turn, and the publisher full text was inaccessible. Verify its methods before reproducing them. [S1, S2, R1]

Research position and novelty boundaries

Prior work	What already exists	Implication for iGAZE
GeoGaze, 2026 [R2]	MediaPipe landmarks and normalized iris ratios for categorical gaze; CPU operation.	Do not claim iris ratios or training-free directional gaze as new.
GazeFollower, 2025 [R3]	Open Python webcam gaze, calibration, filtering and personalization; evaluated with 31 volunteers.	A GUI, logging or local webcam alone is product engineering, not research novelty.
EyeTheia, 2026 preprint [R4]	Landmarks plus CNN, user adaptation and experimental comparison with SeeSo.	Personal calibration is established; specify the new decision policy and evaluate it.
LookAHead, 2026 [R1]	Gaze jump plus head refinement and implicit calibration, according to S1.	Hybrid pointing itself is prior art. Blink gating alone needs a stronger comparative case.
Global South prototype, 2026 [R5]	MediaPipe/OpenCV/PyAutoGUI, gaze/head control, dwell click and exploratory testing with motor-disability users.	Low-cost accessibility and a developing-region setting are not sufficient novelty claims.
EMC-Gaze, 2026 preprint [R6]	Session adaptation, landmark geometry and calibration-aware deployment.	Address drift with measured validation; avoid claiming adaptation as wholly new.
Error prevention in VR, 2026 preprint [R7]	Anomaly-based filtering of unintended gaze selections with 41 participants.	Error prevention is an active field. VR outcomes do not establish desktop performance.
Chiasm, 3 Nov 2026 preprint [R8]	505-person in-the-wild validation; accuracy, precision, eyewear and drift.	Measure environmental robustness, invalid data and calibration drift, not just headline accuracy.

Research conclusion: pursue a practical contribution in reliable, intent-controlled desktop tasks, with component ablations and a relevant user population. Research novelty is provisional until a targeted literature review and experiments establish the exact advance. This is a focused scan, not an exhaustive systematic review.

Accuracy in centimetres, directional classification percentages, event precision and FPS measure different things. They cannot be ranked as a common leaderboard. Sources were checked on 7 November 2026; preprints are explicitly identified.

The proposed interaction contribution

Intent and quality control policy

Implement an explicit state machine: STOPPED, CALIBRATING, READY, ARMED, PENDING, EXECUTING, PAUSED and TRACKING_LOST. Ordinary reading produces no discrete navigation until the selected profile permits a deliberate gaze hold or command-menu confirmation. Continuous scrolling is active only in its dedicated state.

• **Observe:** timestamp each frame using a monotonic clock; derive eye openness, pose, iris ratios and validity indicators. Distinguish unknown from eyes closed.

• **Qualify:** reject missing/stale faces, invalid eye geometry and implausible pose. Brightness is a warning cue, not a calibrated probability of accuracy. Do not invent per-frame confidence if the API does not expose it.

• **Arm:** require a deliberate calibrated trigger or explicit UI activation. Offer a simple dwell alternative if triple blinks are unreliable. No normal blink should click by default.

• **Preview:** show the command name and countdown. On cancellation, face loss or application change, discard the pending action. Require re-arming after reacquisition.

• **Execute:** validate the foreground window immediately before dispatch. Issue at most one discrete action, enter cooldown and require return to neutral before repeating.

• **Recover:** allow inverse navigation for a small allowlist. Do not advertise universal undo: a click, app launch or external edit may be irreversible.

A concrete exhibition example

In Reading mode, the user opens a large command menu, looks right to select Next tab, sees a short pending indicator, then confirms. Looking away cancels. Head tilt scrolls only after Scroll mode is explicitly engaged. In Slides mode, the same signals mean Previous slide and Next slide. Unknown applications default to paused or a manually chosen profile.

Research hypotheses to test after implementation

H1: quality rejection and neutral-return gating reduce unintended commands during reading. H2: context profiles reduce mode errors compared with fixed mappings. H3: explicit correction with bounded calibration updates reduces session drift without increasing false activations. H4: a hybrid pointer reduces required head movement for some target layouts; faster performance is not assumed.

Implement H1/H2 now and record the required evidence. H3/H4 are extension studies. A feature combination may improve usability without establishing methodological novelty; publication claims must follow actual measurements.

Required features and completion criteria

ID and feature	What will ship	Done when
F01 Tracking safety	Face-loss freeze; stale-frame timeout; deliberate recovery; visible stop and emergency pause.	Zero OS events on missing/stale input in replay; 20 stop/start cycles without overlapping workers.
F02 Calibration wizard	Camera check, open/closed samples, neutral pose, left/right range and validation; save personal profile.	Bad/insufficient samples rejected; new calibration runs even if a profile exists; restart reloads valid parameters.
F03 Intent commands	Pending action overlay, neutral return, cooldown, confirm/cancel; retain triple blink as configurable option.	Ordinary reading/blinks do not click; only a valid armed sequence dispatches once.
F04 Everyday navigation	Browser tab actions, proportional scroll and presentation next/previous. App switching only if stable.	Three-minute hands-free workflow passes five consecutive runs on exhibition hardware.
F05 Context profiles	Reading and Slides profiles, manual override and foreground-window check.	Changing app cancels pending actions; unsupported apps are clearly indicated.
F06 Desktop redesign	Resizable PySide6 dashboard, integrated camera preview, wizard, profiles, activity and settings.	Usable at 100/125/150 percent Windows scaling; no clipped controls; keyboard-accessible pause.
F07 Persistent local data	SQLite profiles, calibration history, sessions, action events and CSV export.	Settings survive restart; export totals reconcile with DB; user can delete their history.
F08 Distribution	Offline Windows x64 installer, versioned portable backup and model asset.	Fresh Windows account/machine launches without Python or internet; uninstall works.
F09 Product website	Home, About, Features, Download; real screenshots and release metadata.	Responsive pages; real installer link works; hash matches; no invented claims or placeholder downloads.

Priority order: F01-F04 correctness, F06 usability and packaging proof, F05/F07 integration, then F08/F09 release completion. UI and installer work start early to expose integration problems; final visual polish comes after working flows.

Stretch scope and deliberate deferrals

Feature	Small feasible version	Release rule
Hybrid pointer	9-point calibration with held-out targets; gaze selects a broad region; relative yaw/pitch refines; deliberate click trigger.	Maximum 3 hours on Day 2 only after core milestones. Hide if large-target validation fails.
1-Euro filter	Time-aware filtering compared with existing EMA using identical replay.	Keep only if jitter improves without unacceptable delay. No automatic claim of superiority.
Expression-robust pose	Evaluate a stable eye/nose/ear landmark set with consistent canonical 3D correspondences.	Do not simply replace point IDs in the old PnP model. Compare smiling/talking with neutral-pose baseline.
Alternative trigger	Calibrated right wink or eyebrow raise, selected per user.	Choose one optional trigger; reject asymmetric-eye noise. Voice is deferred.
Session summary	Actions, active minutes, cancellations, quality-loss time and task outcomes.	Required aggregate storage; optional polished charts if time remains.
Presentation extras	Local slide deck, gesture tutorial and audio feedback.	Next/previous required; volume and media controls optional and app-scoped.

After 12 November

• Online gaze adaptation with independent click labels, region-balanced replay, held-out validation, bounded updates and rollback. System-generated gaze clicks are never ground truth.

• A learned intent verifier using genuinely labelled accepted/rejected events. An immediate undo is only a noisy signal, not proof that every other action was correct.

• Fine-grained pointer control, drag/drop, virtual keyboard, extensive app adapters and multi-monitor calibration.

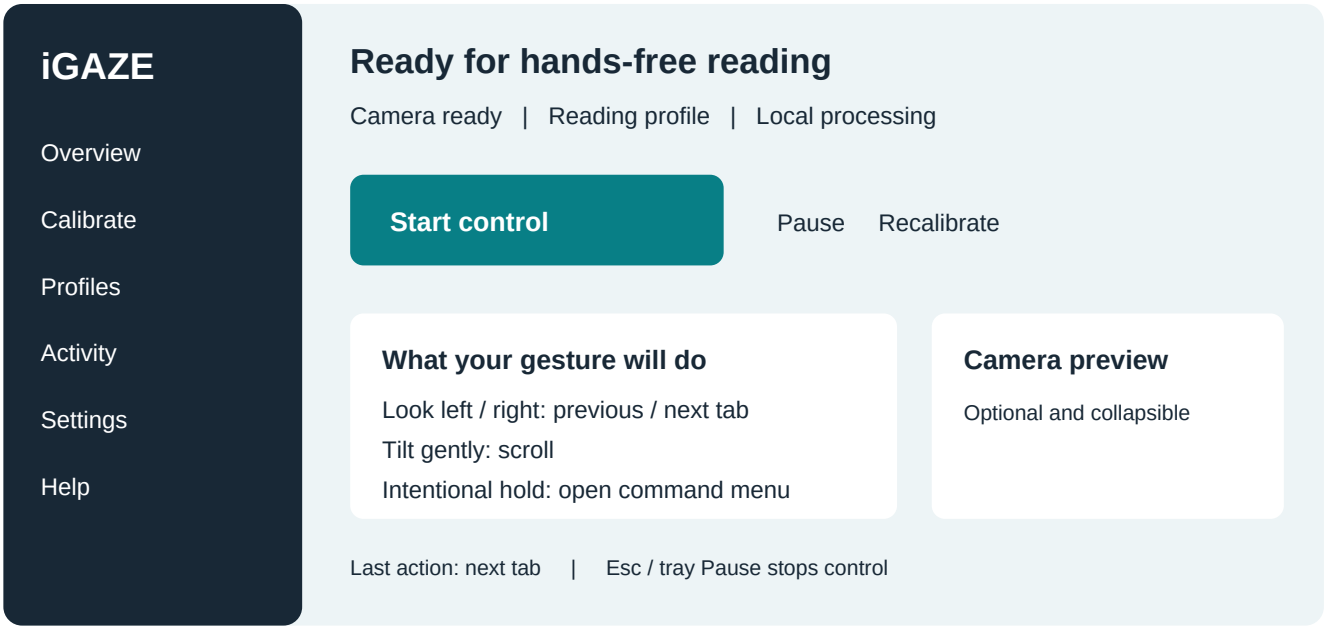
• Formal eyewear and target-user study, longitudinal fatigue/drift evaluation and paper revision.

Avoid in the four-day release

Training a new deep gaze model, cloud accounts, social login, cloud analytics, autonomous LLM desktop control, Android APK, macOS/Linux packaging and an auto-updater. These add dependencies without proving the core interaction. Windows requires an EXE installer or MSI, not an Android APK.

Desktop UI and UX specification

Recommended UI: PySide6 Qt Widgets with a small shared design system. Preserve the Python vision core and replace the Tkinter presentation layer. Prove a minimal bundled Qt window with MediaPipe on Day 1 before completing the redesign. If this spike fails, restyle Tkinter for the exhibition and schedule Qt migration afterward.



Six screens with one clear primary action

Screen	User-facing content
Overview	Start/Pause, active profile, tracking quality, pending command, last action and collapsible preview. Technical landmark overlays are hidden by default.
Calibration	Camera picker, face positioning, sufficient open/closed samples, neutral pose, directional targets, independent validation and Retry/Save.
Profiles	Reading, Slides and custom profile; editable command map; trigger preview; test mode that does not control the OS.
Activity	Session timeline, counts, cancellation/quality reasons, export and delete. Do not label action count as productivity gained.
Settings and Help	Camera, sensitivity, local storage, sound, pause shortcut, startup choice, tutorial and troubleshooting.

Visual system: restrained teal accent, warm white and dark navy surfaces, consistent spacing and 16-18 px body text. Minimum 44 px primary controls, visible focus, text plus colour for statuses, reduced motion, no blink-only setup requirement. English first; Urdu guidance can follow verified core flows.

Code architecture and folder structure

Refactor incrementally behind typed interfaces. Extract current algorithms before changing their behavior. One worker owns camera and FaceLandmarker lifecycle; UI stays on the main thread. Use queued Qt signals for immutable snapshots and a command queue for settings. A bounded latest-frame buffer prevents latency growth.

```
igaze/
  pyproject.toml             # pinned compatible dependencies and entry point
  src/igaze/
    app.py                  # composition and lifecycle
    domain/                 # FrameSignals, Intent, Action, Profile, AppState
    vision/                 # camera, landmarks, EAR, gaze, pose, filters
    interaction/            # state machine, intent gate, profiles, cooldown
    actions/                # Windows adapter, dry-run adapter, safe inverse map
    calibration/            # collection, validation, versioned parameters
    storage/                # SQLite repository, migrations, export
    ui/                     # windows, screens, widgets, theme, overlays
    diagnostics/            # timings, replay, structured logs
    resources/              # model, icons, sounds, license notices
  tests/                    # pure logic, replay, storage and integration
  packaging/                # PyInstaller spec and Inno Setup script
  website/                  # static product site
  docs/                     # build, demo, research protocol and release notes
  scripts/                  # reproducible build and validation commands
```

Non-negotiable engineering changes

No import-time desktop-control side effects. Start/stop are idempotent; stop joins workers before releasing resources. Use try/finally cleanup and report failures to the UI. Face loss resets blink, gaze dwell and scroll state. Use elapsed milliseconds instead of frame counts for gesture timing. Cap scroll integration time after stalls and reset on pause/resume.

PyAutoGUI scroll values are wheel steps, not pixels. Rename the current pixel variables or implement a tested Windows scroll adapter with explicit units. Avoid simultaneously enabling scroll and head-refined pointing. Resource lookup must work from source and packaged builds, independent of the current directory.

Data storage and research-ready instrumentation

Use SQLite in the user application-data directory, for example %LOCALAPPDATA%/iGAZE. It needs no server. Do not write settings next to the installed executable or inside Program Files. Store schema versions and migrate predictably; retain the old JSON calibration as an importable legacy profile, subject to revalidation.

Entity	Core fields	Purpose
profiles	id, nickname, app rules, trigger policy, settings version	Personal configuration without collecting diagnosis or real identity.
calibrations	profile id, timestamp, algorithm/model version, parameters, sample counts, validation score	Reproduce a run and roll back an invalid update.
sessions	id, profile id, start/end, build id, camera resolution, device summary	Join outcomes to the exact runtime conditions.
action_events	session id, monotonic offset, candidate/confirmed/executed/cancelled, reason, action, app category	Measure the interaction pipeline and accidental-action labels.
quality_samples	session id, timestamp, validity reason, pose/gaze summary, frame age	Low-rate diagnostic sampling; disabled unless enabled.
task_trials	task id, condition, target, start/end, outcome, errors, annotator source	Reproducible benchmarks and participant-level analyses later.

Storage boundaries

• Default: calibration/settings and aggregate activity. Detailed research event logging is opt-in. Diagnostic recording is separately enabled with visible status and a retention limit.

• Raw video and face images are off by default. Do not persist window titles, URLs, document text or screenshots. Use app categories and pseudonymous session IDs.

• One storage writer batches queued records outside the capture loop. Bound the queue; expose dropped-record counts. Logging failure must not block Pause or crash tracking.

• Support Export session CSV, Export diagnostics, Delete selected session and Delete all local data. Warn accurately that local SQLite is not automatically encrypted.

Measurements worth collecting now

Capture-to-decision and decision-to-dispatch timings, frame age, valid-input fraction, calibration retries, cancelled commands and independently labelled unintended actions. OS dispatch success is not proof of task success; add target-state observation or a user outcome label. FPS alone does not measure end-to-end response time.

Windows installer and download pipeline

Deliver iGAZE-Setup-0.4.0-win-x64.exe as the proposed exhibition version and an optional portable ZIP. The installer bundles Python, compatible native libraries, MediaPipe model, UI assets and required runtimes. The user should not run pip, download a model or install Python. Version number is proposed until a release is built.

Stage	Implementation and verification
Dependency proof	Use a clean Windows x64 virtual environment with a tested Python version. Do not assume the README Python 3.12+ claim is verified: the existing venv path shows Python 3.10. Pin a compatible interpreter/package set after smoke testing.
Freeze	Start with a PyInstaller one-folder build and explicit model/data/native-library collection. Test MediaPipe and Qt plugin paths. Qt pyside6-deploy/Nuitka is a fallback if the first spike fails. [T1, T2]
Install	Wrap the known-good folder in Inno Setup. Per-user installation where possible, shortcuts, version metadata, uninstall entry and third-party notices. Preserve user data unless deletion is chosen. [T3]
Validate	Install on a clean Windows user or second laptop with no Python and no internet. Check camera permission denial, missing camera, spaces/non-ASCII paths, restart and uninstall.
Publish release	Create a versioned release asset, SHA-256 checksum, actual size, supported OS/architecture, changelog and known limitations. Prefer the repository release host; website points to that exact asset.
Download contract	Website release metadata includes version, filename, URL, size, hash and release date. Verify HTTPS download and hash from another device. Never ship a dead or placeholder Download button.

Packaging risks to handle early

Native DLLs, Qt plugins and MediaPipe assets can be omitted by automatic freezing. Cold-start may differ from the source environment. Build first on Day 1 and repeat only when packaging inputs change. If installer integration fails late, distribute a verified portable folder ZIP and label it accurately; do not call it an installer.

Unsigned Windows builds can display trust warnings. Explain the actual publisher/signing status without telling users to disable protection. Review model redistribution terms and PySide6/other dependency obligations before release. Code signing and domain expenses are optional decisions; this plan assumes no purchase.

Website product and content plan

Build a static responsive website using Astro with minimal JavaScript, shared design tokens and reusable components. A static site avoids maintaining an account system or backend. Use actual app screenshots/video after the desktop flows work; the marketing site must reflect the released build.

Page	Required sections	Primary action
Home	Clear hands-free reading/navigation promise; real demo clip; three use cases; how setup works; local-processing explanation; supported Windows release.	Download for Windows
About	Problem, intended users, NED team details only as supplied, design principles, research status and honest limitations.	View features / try tutorial
Features	Reading, Slides, command preview/cancel, calibration, profiles, activity and privacy controls. Label experiments separately from stable features.	See it working
Download	Verified EXE, version/size/architecture, system requirements, checksum, install guide, portable backup, changelog and troubleshooting.	Download exact release asset

Visual direction

A clean assistive-tech identity: generous white space, navy typography, restrained teal, consistent eye-inspired mark, accessible contrast, real software imagery and short copy. Use small transitions with reduced-motion support. Avoid invented medical endorsements, fake testimonials, fake adoption counts and unmeasured accuracy badges.

Page behavior and accessibility

Responsive at 360 px mobile and desktop widths. Semantic headings, keyboard navigation, visible focus, accessible download labels, alt text, video captions/transcript and compressed media. Camera access is unnecessary on the marketing website. Avoid analytics/cookies for the first release unless there is a concrete need.

Hosting and release connection

Use a static host already available to the team, with a free subdomain if needed. Keep binaries on the release host rather than in the website Git repository. Website reads a checked-in release manifest; automatic update infrastructure is deferred. Until the asset is verified, preview shows Release not yet available instead of pretending a download exists.

Website acceptance: four pages reachable, no broken internal links, production HTTPS, correct metadata and page titles, tested mobile layout, and an end-to-end download/install on a second machine. The public site may be prepared locally while release packaging is in progress.

Four-day schedule with scope gates

Planning dates: Sunday 8 November through Wednesday 11 November 2026. Exhibition: Thursday 12 November. Use any remaining 7 November time for repository backup, task setup and collecting hardware/access details. All deadlines below use Asia/Karachi time.

Date and budget	Work blocks	Exit gate
8 Nov / 8 hours	1.5h package extraction and typed signals; 3h lifecycle, validity and scroll fixes; 2h calibration failures/retry; 1.5h minimal PySide6 + MediaPipe packaged proof.	No stale-input actions; stop/start works; fresh calibration works; installed prototype opens. If Qt proof fails, keep/restyle Tkinter.
9 Nov / 8 hours	2h intent states, pending overlay and neutral return; 2h Reading/Slides profiles; 3h desktop screens and SQLite core; 1h replay and export checks.	Three tasks usable; settings persist; candidate and executed events export correctly. Pointer stretch permitted only if all are done early.
10 Nov / 8 hours	3h website pages; 2h dashboard/profile polish and accessible layout; 2h installer/resource validation on second machine; 1h actual screenshots and tutorial.	Website preview complete; offline installer works; no core flow depends on an experimental feature.
11 Nov / 8 hours	2h integration fixes; 2h gesture/lighting/eyewear and 30-minute soak checks; 1h installer freeze and release asset; 1h production website/download test; 2h rehearsal and contingency.	Feature freeze by noon, defect-only work afterward. Release and backup ready by evening. No new feature after freeze.

If time or quality slips

Cut in this order: hybrid pointer, automatic adaptation, optional trigger, advanced charts, automatic context detection (keep manual profiles), App Switcher (keep reliable tab and slide commands). Preserve Pause, calibration quality, core tasks, usable UI, basic local storage and a working distribution path.

Critical dependency chain

Typed vision signals -> validity/lifecycle -> intent state machine -> app actions -> UI/storage -> packaged release -> website download -> rehearsal. Website static content can be prepared earlier; its Download button is finalized only after the executable passes testing.

Pointer exploration is outside the 32-hour baseline unless required work finishes early. Do not silently absorb its effort into the required schedule. Several weeks of research work in S1 cannot be promised within four days solely because AI writes code quickly.

Testing and exhibition readiness

The following are release targets, not observed results. Record actual outcomes on the exhibition machine. If a gate fails, fix or remove the affected feature from the stable demo.

Gate	Verification
No stale control	Replay no-face, camera stall, eye occlusion and reacquisition sequences. Assert zero new OS actions while invalid; require explicit re-arming.
Correct lifecycle	Run 20 start/stop cycles, close during calibration, unplug/reconnect camera and relaunch. No duplicate worker, camera lock or unhandled crash.
Gesture intent	Read naturally for five minutes with a human observer, then run 20 intended actions per enabled gesture. Target <=1 unintended command in five minutes and >=90 percent intended success; report exact counts.
Calibration	Test insufficient samples, missing face, old/corrupt file and profile changes. Reject invalid values; retain last known-good calibration; no silent success.
Performance	Target >=20 median FPS on exhibition hardware; show p95 frame age and capture-to-dispatch timing separately. Drop old frames instead of queueing delayed actions.
Storage	Restart persistence, export consistency, logging off, schema migration and delete controls. Simulate a failed write; Pause still works.
UI and package	100/125/150 percent scaling, keyboard traversal, camera permissions and clean Windows installation without Python. Website download matches published hash.
Soak and rehearsal	30 minutes without crash and five consecutive three-minute demos. Keep the exact tested binary, profiles, local files and presentation deck frozen.

Three-minute demonstration

0:00 explain the restricted-hand use case and show ordinary webcam/local processing. 0:20 calibrate or load a visibly validated personal profile. 0:50 read and scroll a local document, then deliberately change tabs. 1:40 change to Slides and navigate a deck. 2:10 show tracking-loss pause/cancel and activity history. 2:40 show the website and real installer. Explain any experimental pointer separately.

Fallback kit

Carry the tested installer, portable folder ZIP, local PDF/slides, known-good calibration, spare webcam if available, power adapter and a clearly labelled recorded demo. The recording is a contingency, never presented as live. Disable disruptive notifications and rehearse under exhibition-like lighting.

Evidence for the paper after the exhibition

Build traceable evidence now; rewrite the paper after the product stabilizes. Old manuscript results remain unverified until the original participant-level records and plotting files are checked. Do not republish illustrative graphs or reconstruct missing demographics and p-values.

Research question	Comparison	Data required
Does quality/intent control reduce unintended commands?	Old fixed rule pipeline vs new quality gate vs quality + intent policy.	Reading/idle exposure minutes, TP/FP/FN event labels, missed intended commands and recovery time.
Do context profiles help everyday tasks?	Fixed mappings vs Reading/Slides profiles; counterbalanced task order.	Time, mode errors, success, cancellations and user workload.
Does hybrid pointing reduce effort?	Head-only vs gaze-only where feasible vs hybrid; same target task.	Target distance/size, completion time, misses and an independently defined head-motion measure.
How robust is the system?	Measured normal/dim/backlit conditions, glasses strata, camera distance and session changes.	Validity rate, error, calibration failures and drift. No invented angular error from five-class labels.
Who benefits?	Relevant target users plus appropriately described comparison sample.	Consent, inclusion criteria, observed functional tasks, SUS and NASA-TLX; no injury-prevention claim.

Reproducibility checklist to implement now

Store software commit/build, algorithm settings, model checksum, OS/device/camera details, calibration version and task condition with each session. Preserve unaggregated task records under pseudonymous IDs; provide an export dictionary with units and metric denominators. Keep test data separate from tuning/calibration data.

Post-exhibition sequence

13-15 November: audit old data and freeze a reproducible baseline. Next: finalize a consented evaluation protocol with the supervisor, pilot tasks and select a justified participant/sample plan. Then collect data, run component ablations and appropriate within-subject analyses, report effect sizes/confidence intervals, and choose the submission venue to fit the evidence.

Formal target-user recruitment depends on institutional review and access. A small exhibition demonstration is feasibility evidence, not clinical validation. Neither a new feature nor polished English guarantees acceptance; a strong paper needs a defensible contribution and reproducible results.

Model allocation and implementation tickets

Use GPT-6.1 Sol at Medium as the default implementation model. Increase effort only for difficult state/lifecycle or integration bugs. Use Luna for bounded tasks with explicit inputs and acceptance checks. Reserve Astra for one architecture/novelty review and, only if needed, a difficult unresolved blocker. The allocation below is a planning policy, not a measured credit-saving guarantee. [M1]

Work package	Model and effort	Review condition
Vision safety, lifecycle, intent engine	Sol Medium; High for difficult failures	Replay tests and no unintended OS dispatch.
Desktop UI, storage and packaging	Sol Medium	Actual rendered app, DB persistence and fresh-machine install.
Website with complete design brief	Sol Medium; Luna for copy/link corrections	Visual inspection on desktop/mobile plus actual download.
Docs, release manifest, fixture preparation	Luna Low/Medium	Check supplied facts; run exact validation commands.
Novelty tradeoff or architecture audit	Astra Medium, one bounded review	Return actionable findings against this plan, no broad rewrite.
Final regression and release fixes	Sol Medium	Only failures or evidence justify expanding the change.

Keep credit use predictable

Aim to spend most implementation interactions on Sol, with small Luna tasks and at most one or two bounded Astra sessions. Exact credit cost depends on your account, model availability and work length; API prices are not a conversion for desktop credits. Choose the lightest model that passes the same acceptance test. Do not use Max/Ultra by default.

Ordered ticket queue

T01 baseline/run instructions and package proof; **T02** typed signals and lifecycle; **T03** quality gate and pause; **T04** calibration wizard; **T05** intent state machine; **T06** Reading/Slides actions; **T07** UI shell/screens; **T08** SQLite/export; **T09** website; **T10** installer; **T11** QA/rehearsal; **T12** hybrid prototype after gates.

Reusable task brief

"Implement ticket Txx from the iGAZE plan. Read the current code first. Preserve working behavior outside scope. Change only the named modules. Add meaningful checks for failure cases. Run the specified acceptance checks and report what passed, what remains and the exact files changed. Stop before adding deferred features."

Execute one integrated ticket at a time; keep a short handoff with interfaces, decisions and passing commands. No additional agent sessions are required by this plan.

Sources and verification notes

Local evidence and supervisor inputs

[S1] iGAZE_Features_Research_Gaps_Report.pdf, six pages, prepared 2 November 2026. Supervisor-provided synthesis, especially LookAHead findings and gaps G1-G9.
[S2] iGAZE_critical_review_and_blue_revisions.docx. Full paragraphs and tables inspected; no comments part. Blue passages are proposed manuscript revisions.
[S3] Original 16-page iGAZE manuscript and seven Python modules/README inspected in this chat. Existing numeric claims are not newly validated by this plan.

[R1] Bui, Nguyen and Ma. LookAHead. PACM HCI, 2026. Detailed methods/results attributed to S1; publisher full text inaccessible in this scan.
https://doi.org/10.1145/3806018

[R2] Khalid et al. GeoGaze. ICCK Transactions on Advanced Computing and Systems 2(2), 2026.
https://doi.org/10.62762/TACS.2025.798133

[R3] Zhu et al. GazeFollower. PACM Computer Graphics and Interactive Techniques 8(2), 2025. Primary publisher text retrieved in prior scan.
https://doi.org/10.1145/3729410

[R4] Pather et al. EyeTheia. arXiv v2, June 2026; ICPR 2026 proceedings citation carries 2027 volume metadata.
https://arxiv.org/abs/2601.06279

[R5] Gonzalez Maldonado et al. Accessible Eye-Tracking in the Global South. Engineering Proceedings 153(1), 24, 2026. Publisher text and university record checked in prior scan.
https://www.mdpi.com/2673-4591/153/1/24

[R6] Deployment-Oriented Session-wise Meta-Calibration for Landmark-Based Webcam Gaze Tracking. 2026 preprint.
https://arxiv.org/abs/2603.12388

[R7] Severitt et al. A Real-Time Error Prevention System for Gaze-Based Interaction in Virtual Reality Based on Anomaly Detection. 2026 preprint.
https://arxiv.org/abs/2601.15146

[R8] Belopolsky and Perdreau. Chiasm in-the-wild validation and comparison with WebGazer. 3 November 2026 preprint.
https://arxiv.org/abs/2611.04946

[T1] PyInstaller documentation. Self-contained folder/executable distribution including the interpreter.
https://pyinstaller.org/en/stable/operating-mode.html

[T2] Qt for Python deployment. PySide6 deploy/Nuitka and standalone packaging.
https://doc.qt.io/qtforpython-6/deployment/deployment-pyside6-deploy.html

[T3] Inno Setup. Windows installer builder; review applicable redistribution/use terms.
https://jrsoftware.org/isinfo.php

[M1] OpenAI official model-selection guidance. Sol for complex work with cost constraints; Luna for scoped efficient tasks; Astra for demanding work.
https://developers.openai.com/api/docs/guides/model-selection

Research access date: 7 November 2026. Source reports, preprints and planning recommendations have different evidential status. No unpublished iGAZE performance, medical outcome or research acceptance is implied.